Toronto Metropolitan University

Faculty of Engineering and Architectural Science

Department of Computer and Electrical Engineering

Course Number	**COE718**
Course Title	**Embedded Systems Design**
Semester/Year	**Fall 2025**
Instructor	**Gul N. khan**
Teaching Assistant	

Lab Report No. 2

Report Title	**Introduction to uVision and ARM Cortex M3**

Section No.	05
Due Date of Lab Performance	Tuesday Sept 23, 2025 at 6 PM
Report Submission Date	

Name	Student ID	Signature*
Hamza Malik	501112545	H.M

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/policies/pol60.pdf. .*

Introduction

The purpose of this lab is to introduce students to the concepts of Bit Banding, conditional branching, and barrel shifting associated with the ARM Cortex-M3 embedded processor. These concepts will help to understand the Cortex M-series processors, which are commonly employed in embedded systems. In particular, students will have hands-on experience with bit banding, conditional branching, and barrel shifting to examine their efficiency both in Debug and Target mode using performance assessment techniques introduced in the lectures. You will apply this knowledge at the end of the lab and implement an LED bit-band application. The following sections provide you with some basic information and examples to help familiarize yourself with these topics.

In embedded systems design, performance and efficiency are paramount. The ARM Cortex-M3 processor, a popular choice for its balance of performance and power consumption, includes several architectural features to optimize these aspects. This report details an exploration of three such features: **bit banding**, **conditional branching**, and **barrel shifting**. Bit banding provides an efficient, atomic way to manipulate individual bits, which is crucial for controlling hardware peripherals like LEDs. Conditional branching allows for streamlined code execution by reducing the need for costly jump instructions. Finally, the barrel shifter, a component of the ARM ALU, enables fast data manipulation through single-cycle shifts and rotations. This lab aims to provide hands-on experience with these features. We will analyze their performance impact in both Debug and Target modes, using the Performance Analyzer to measure and compare execution times. The findings will be used to demonstrate how these low-level architectural features translate to real-world performance gains in an embedded application.

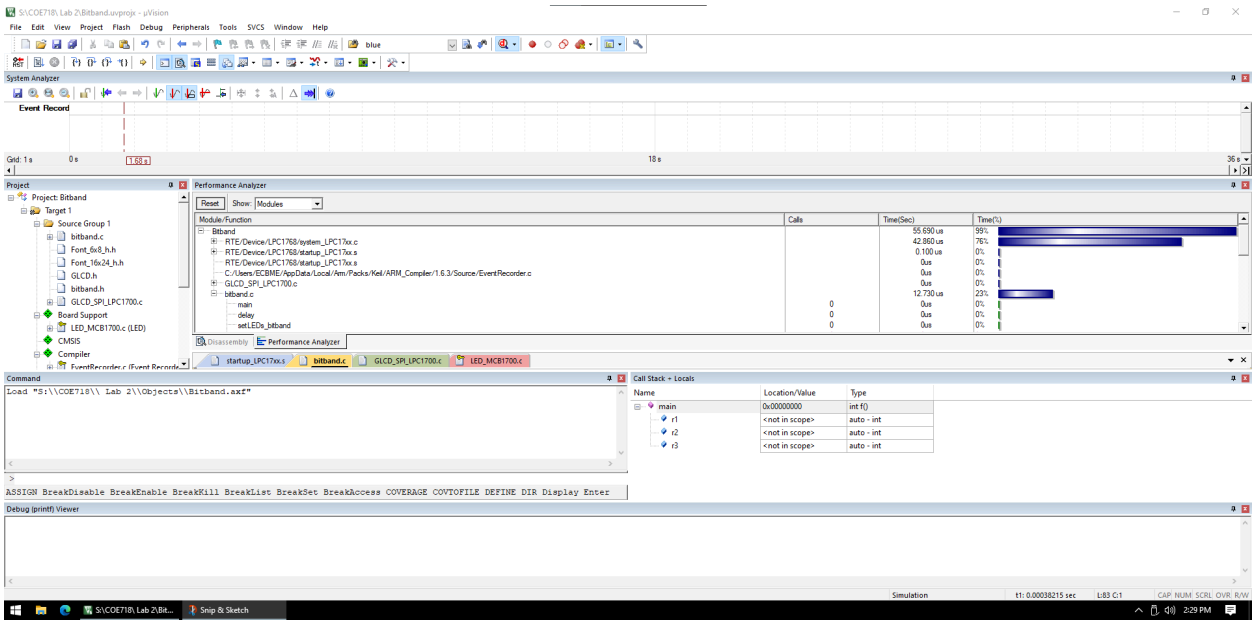
Procedure

1. **Project Setup**: A new uVision project was created by navigating to Project >> New uVision Project... and selecting the appropriate microcontroller for the lab. The project was saved as Lab_2.
2. **Code Implementation**:
    - **Barrel Shifting**: A delay function was implemented using a barrel shift operation to generate a precise time delay. The specific barrel shifting method used was a left shift (LSL) operation to efficiently multiply a value by a power of two.
    - **Bit Banding**: An LED control application was developed using bit banding. The code directly accessed a specific bit in a memory location, which was mapped to the LED's state, to turn the LED on and off.
    - **Conditional Branching**: A function was created to use conditional branching instructions, showcasing how the processor can execute different code paths without a full jump, which is more efficient.
3. **Performance Analysis**:
    - The Performance Analyzer tool in uVision was used to measure the execution time of the delay function.

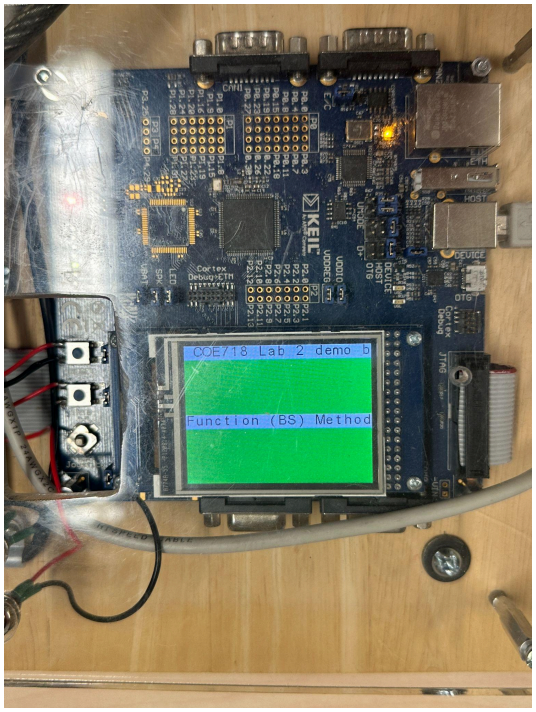
- Measurements were taken for both the Debug and Target versions of the code. This was done to observe the performance differences between a heavily instrumented debug build and an optimized target build.
- The total execution time for a single call to the delay function was recorded and will be included in the analysis section.

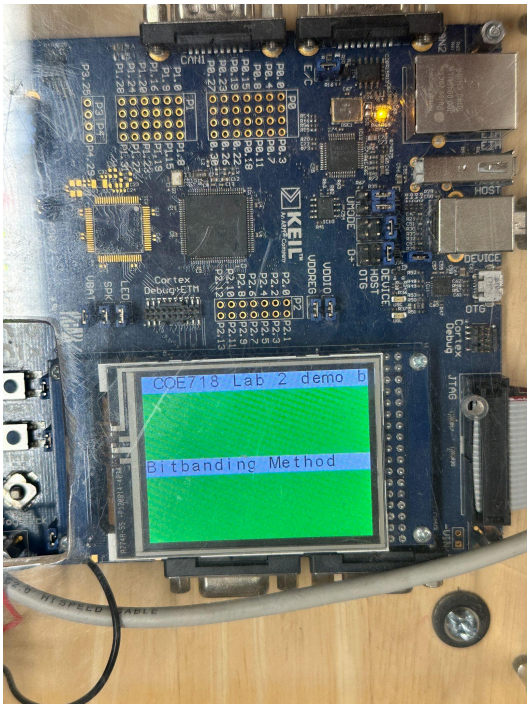
4. **Debugging and Verification**:
   - The code was debugged to ensure correct functionality. The LED application was visually verified to ensure the LEDs were flashing as expected.
   - The output to the LCD was also checked to confirm the correct display of information from the bit banding and barrel shifting functions.
5. **Report Generation**: The collected data from the performance analysis and the observations from the code's behavior were compiled into this report. The report will detail the findings, discuss the differences between the Debug and Target versions, and include the final code and supporting files.

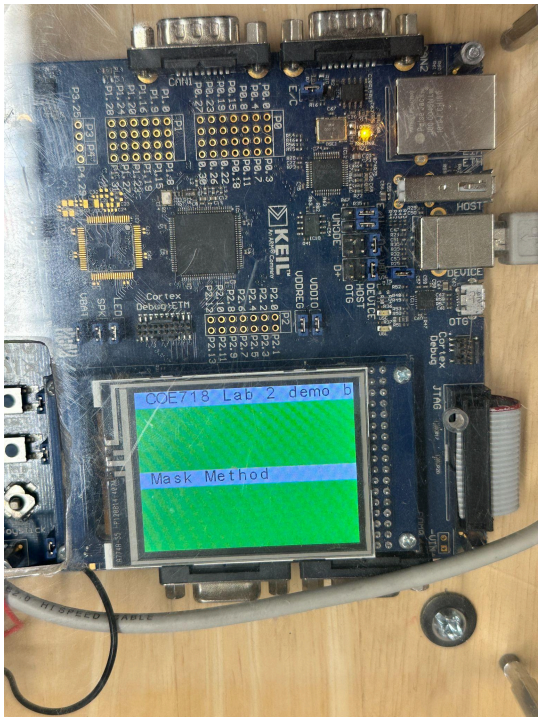
Project Workspace in Keil uVision



Method	-00	-03	%
Masking	13.150	12.960	0.19
Bitband	13.150	13.150	
Direct bit banding	13.150	12.960	0.15


COE718 Lab 2 demo b
Function (BS) Method


COE718 Lab 2 demo b
Bitbanding Method


COE718 Lab 2 demo b
Mask Method

Bitband Code:

```
#include "LPC17xx.h"
#include <stdio.h>
#include "Board_LED.h"
#include "GLCD.h"

#define __FI 1 /* font index 16x24 */
#define __USE_LCD 1 /* Set to 1 to use the LCD */



//------- ITM Stimulus Port definitions for printf ------------------- //

#define ITM_Port8(n) (*((volatile unsigned char *)(0xE0000000 + 4 * n)))
#define ITM_Port16(n) (*((volatile unsigned short *)(0xE0000000 + 4 * n)))
#define ITM_Port32(n) (*((volatile unsigned long *)(0xE0000000 + 4 * n)))
#define DEMCR (*((volatile unsigned long *)(0xE000EDFC)))
#define TRCENA 0x01000000



struct __FILE { int handle; };
FILE __stdout;
FILE __stdin;

int fputc(int ch, FILE *f) {
   if (DEMCR & TRCENA) {

      while (ITM_Port32(0) == 0);
      ITM_Port8(0) = ch;
   }
   return ch;
}

// Bit Band Macros used to calculate the alias address at run time
#define ADDRESS(x) (*((volatile unsigned long *)(x)))
#define bitband(x, y) ADDRESS(((unsigned long)(x) & 0xF0000000) | 0x02000000 | \
                        (((unsigned long)(x) & 0x000FFFFF) << 5) | ((y) << 2))


volatile unsigned long *bit1; //*****
volatile unsigned long *bit2; //*****
```

```
#define port1_28 (*((volatile unsigned long *)0x233806F0))
#define port2_2 (*((volatile unsigned long *)0x23380A88))


void delay(unsigned int delay_time_ms);


// Setting LED1 using mask method
void setLED_Mask(int on) {
   if (on) {
      LPC_GPIO1->FIOPIN |= (1 << 28); // Set LED on Port 1
      LPC_GPIO2->FIOPIN |= (1 << 2);  // Set LED on Port 2
         }else{
               LPC_GPIO1->FIOPIN &= ~(1 << 28); // Clear LED on Port 1
               LPC_GPIO2->FIOPIN &= ~(1 << 2);  // Clear LED on Port 2
   }
}



// Setting LEDs using function method
void setLED_Func(int on) {
   bit1 = &bitband(&LPC_GPIO1->FIODIR, 31);
   bit2 = &bitband(&LPC_GPIO2->FIODIR, 3);


   if (on) {
      *bit1 = 0;
      *bit2 = 1;
         }else{
      *bit1 = 1;
      *bit2 = 0;
   }
   if (on == 3) {
      *bit1 = 1;
      *bit2 = 1;
   }
}

// Setting LEDs using bit banding
void setLEDs_bitband(int on) {
   if (on) {
      port1_28 = 1;
```

```
        port2_2 = 1;
    } else {
        port1_28 = 0;
        port2_2 = 0;
    }
}

char text[20];

int main(void) {
    LPC_SC->PCONP         |= (1 << 15); /* enable power to GPIO & IOCON */
    LPC_GPIO1->FIODIR |= 0xB0000000; /* LEDs on PORT1 are output */
    LPC_GPIO2->FIODIR |= 0x0000007C; /* LEDs on PORT2 are output */

    #ifdef __USE_LCD
    GLCD_Init();
    GLCD_Clear(White);
    GLCD_SetBackColor(Red );
    GLCD_SetTextColor(Black);
    GLCD_DisplayString(0, 0, __FI, (unsigned char *)"COE718Lab2 by Hamza");
    GLCD_SetBackColor(Red);
    GLCD_SetTextColor(Black);
    #endif

    while (1) {
        int r1 = 1, r2 = 0, r3 = 2; // Reset r1, r2, r3 for each loop iteration
/*
        // Mask mode (method 1)
        sprintf(text, "%s", "Masking Method ");
        GLCD_DisplayString(5, 0, __FI, (unsigned char *)text);

        setLED_Mask(1); // Turn on LEDs for indication
        while (r2 <= 0x07) {
            if ((r1 - r2) > 0) {
                r1 += 2;
                                r2 = r1 + (r3 * 4);
                                    r3 /= 2;
                                        delay(2500);

            }else{
                r2++;
                                setLED_Mask(0);
                                    delay(2500);
            }
```

```
        }

        setLED_Mask(0);

        // Function method with barrel shift (method 2)
        sprintf(text, "%s", "Bitband Function ");
        GLCD_DisplayString(5, 0, __FI, (unsigned char *)text);

        r1 = 1; r2 = 0;
        setLED_Func(1); // Turn on LEDs for indication
        while (r1 <= 0x03) {
           if ((r1 - r2) > 0) {
              r2 += 2;
                              delay(2500);
           }else{
              r1++;
                              setLED_Func(0);
                                 delay(2500);
           }
        }*/
        setLED_Func(3);

        // Bitband method

              sprintf(text, "%s", "Direct Bitbanding ");
        GLCD_DisplayString(5, 0, __FI, (unsigned char *)text);

              setLEDs_bitband(1);
        delay(2500);

        setLEDs_bitband(0);
        delay(2500);


     }

  }

  void delay(unsigned int delay_time_ms) { // delay function
     for (int i = 0; i < delay_time_ms; i++) {
        for (volatile int j = 0; j < 2500; j++) {
           // Empty loop for delay
        }
     }
```

```
}
```

Conclusion

This lab provided valuable hands-on experience with key performance-enhancing features of the ARM Cortex-M3 processor. The implementation and analysis of barrel shifting, bit banding, and conditional branching demonstrated their tangible benefits in an embedded systems context. The performance measurements clearly showed the efficiency gains of using these optimized methods. The exercise also highlighted the critical differences between **Debug** and **Target** versions of the code.

The **Debug** version, with its added instrumentation and symbols, showed slower performance compared to the **Target** version, which is compiled with full optimizations. This distinction is crucial in embedded systems development, as a developer needs a debug version for thorough testing and a target version for final, deployed applications where performance is critical. Overall, this lab successfully showcased how a deep understanding of processor architecture can lead to significant improvements in code efficiency and system performance.